

Object-Oriented Programming Using Java

Java Language Basics

Contents

- Variables
- Arrays
- Data types
- Operators
- Statements

References:

<https://docs.oracle.com/javase/tutorial/java/nutsandbolts/index.html>

Kinds of variables

- Instance Variables (Non-Static Fields)
- Class Variables (Static Fields)
- Local Variables
- Parameters

Naming Conventions

- Variable names are case-sensitive
- Beginning with a letter, the dollar sign "\$", or the underscore character "_"
- White space is not permitted.
- Variable names must be different to keywords

Naming Conventions

- only one word
 - *all lowercase letters*
- more than one word:
 - *capitalize the first letter of each* subsequent word
- constant value:
 - *capitalizing every letter*

```
int name;
```

```
int currentStudentName;
```

```
static final int NUM_STUDENTS = 6;
```

Primitive Data Types

- A **primitive** is a non-object data type that represents a single value

Data Type	Bytes	Default Value (for fields)
byte	1	0
short	2	0
int	4	0
long	8	0L
float	4	0.0f
double	8	0.0d
char	2	'\u0000' (or 0)
boolean	true/false	false

String – Object Data Type

- String – Object Data Type
- Creating Strings
 - `String name = "Steve";`
 - `String s = "";`
 - `String s = null;`
- String concatenation.
 - `String x = "foo" + "bar" + "!";`

Arrays

- declares an array

```
int[] anArray; //or float anArray[];
```

- allocates memory for array

```
anArray = new int[10];
```

- initialize elements

```
anArray[0] = 10; anArray[1] = 20; //...
```

- the size of any array

```
System.out.println(anArray.length);
```


Operators

Operators	Precedence
postfix	expr++ expr--
unary	++expr --expr +expr -expr ~ !
multiplicative	* / %
additive	+ -
relational	< > <= >= instanceof
equality	== !=
logical AND	&&
logical OR	
ternary	? : <code>int x=100; int result = (x==1) ? 1 : 2;</code>
assignment	= += -= *= /= %=

Statements and Blocks

- A *statement* forms a complete unit of execution
- A *block* is a group of zero or more statements between balanced braces

```
// declaration statement
double aValue = 8933.234;
// assignment statement
aValue = 8933.234;
// increment statement
aValue++;
// method invocation statement
System.out.println("Hello World!");
```

Blocks Demo 1

```
class BlockDemo {  
    public static void main(String[] args) {  
        boolean condition = true;  
        if (condition) { // begin block 1  
            System.out.println("Condition is true.");  
        } // end block 1  
        else { // begin block 2  
            System.out.println("Condition is false.");  
        } // end block 2  
    }  
}
```

Blocks Demo 2

```
1
2  public class HelloWorld {
3      public static void main(String[] args) {
4          int n=10,m=0;
5          if (n > 0){
6              int tong =0;
7              tong +=n;
8          }
9          System.out.println(tong);
10
11         for (int i = 0; i < 10; i++)
12             n+=i;
13             m+=i;
14     }
15 }
```

Statements

- Decision-making statements
 - if-then, if-then-else, switch
- Looping statements
 - for, while, do-while
- Branching statements
 - break, continue, return

if-else Demo

```
class IfElseDemo {  
    public static void main(String[] args) {  
  
        int testscore = 76;  
        char grade;  
  
        if (testscore >= 90) {  
            grade = 'A';  
        } else if (testscore >= 80) {  
            grade = 'B';  
        } else if (testscore >= 70) {  
            grade = 'C';  
        } else if (testscore >= 60) {  
            grade = 'D';  
        } else {  
            grade = 'F';  
        }  
        System.out.println("Grade = " + grade);  
    }  
}
```

switch Demo

```
int month = 8;
String monthString;
switch (month) {
    case 1: monthString = "January";
            break;
    case 2: monthString = "February";
            break;
    case 3: monthString = "March";
            break;
    case 4: monthString = "April";
            break;
    case 5: monthString = "May";
            break;
    case 6: monthString = "June";
            break;
    case 7: monthString = "July";
            break;
    case 8: monthString = "August";
            break;
    case 9: monthString = "September";
            break;
    case 10: monthString = "October";
            break;
    case 11: monthString = "November";
            break;
    case 12: monthString = "December";
            break;
    default: monthString = "Invalid month";
            break;
}
System.out.println(monthString);
```

while and do-while Demo

```
class WhileDemo {  
    public static void main(String[] args){  
        int count = 1;  
        while (count < 11) {  
            System.out.println("Count is: " + count);  
            count++;  
        }  
    }  
}
```

```
class DoWhileDemo {  
    public static void main(String[] args){  
        int count = 1;  
        do {  
            System.out.println("Count is: " + count);  
            count++;  
        } while (count < 11);  
    }  
}
```


for and for-each Demo

```
class ForDemo {  
    public static void main(String[] args){  
        for(int i=1; i<11; i++){  
            System.out.println("Count is: " + i);  
        }  
    }  
}
```

```
class EnhancedForDemo {  
    public static void main(String[] args){  
        int[] numbers =  
            {1,2,3,4,5,6,7,8,9,10};  
        for (int item : numbers) {  
            System.out.println("Count is: " + item);  
        }  
    }  
}
```

break Demo

```
class BreakDemo {  
    public static void main(String[] args) {  
  
        int[] arrayOfInts = { 32, 87, 3, 589, 12, 1076,  
                               2000, 8, 622, 127 };  
        int searchfor = 12;  
  
        int i;  
        boolean foundIt = false;  
  
        for (i = 0; i < arrayOfInts.length; i++) {  
            if (arrayOfInts[i] == searchfor) {  
                foundIt = true;  
                break;  
            }  
        }  
  
        if (foundIt) {  
            System.out.println("Found " + searchfor  
                               + " at index " + i);  
        } else {  
            System.out.println(searchfor  
                               + " not in the array");  
        }  
    }  
}
```

continue Demo

```
class ContinueDemo {  
    public static void main(String[] args) {  
  
        String searchMe = "peter piper picked a " + "peck of pickled peppers";  
        int max = searchMe.length();  
        int numPs = 0;  
  
        for (int i = 0; i < max; i++) {  
            // interested only in p's  
            if (searchMe.charAt(i) != 'p')  
                continue;  
  
            // process p's  
            numPs++;  
        }  
        System.out.println("Found " + numPs + " p's in the string.");  
    }  
}
```

Infinite loop

- using the for statement
`for (; ;) { //code here }`
- using the while statement
`while (true) { //code here }`

Questions and Exercises

- Variables:
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/QandE/questions_variables.html
- Operators:
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/QandE/questions_operators.html
- Expressions, Statements, and Blocks:
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/QandE/questions_expressions.html
- Control Flow Statements:
https://docs.oracle.com/javase/tutorial/java/nutsandbolts/QandE/questions_flow.html